

Developing a Grover’s quantum algorithm emulator on standalone FPGAs — 해설·정리본

항목	내용
원문 제목	Developing a Grover’s quantum algorithm emulator on standalone FPGAs: optimization and imp
저자	Seonghyun Choi, Woojoo Lee (중앙대학교 지능형반도체공학과)
게재지 / 연도	AIMS Mathematics, 9(11): 30939-30971, 2024 (Open Access, CC BY 4.0)
DOI	10.3934/math.20241493
분량	33쪽 (지면 30939-30971)
원문 파일	../papers/2. Grover 논문 (LPSoC)/Developing a Grover's quantum algorithm emulator

이 문서의 쪽 번호는 **PDF 뷰어 기준(1~33쪽)** 입니다. 지면 번호와는 PDF p.N = 지면 30938+N 으로 대응합니다 (예: PDF p.21 = 지면 30959).

한눈에 보기

FPGA로 양자컴퓨팅(QC) 에뮬레이터를 만들면 큐비트 수 n 이 늘 때 상태벡터 크기가 $N = 2^n$ 으로, 게이트 행렬이 $N \times N$ 으로 폭발해서 저가 FPGA에는 아예 안 들어갑니다. 이 논문은 **범용 QC 에뮬레이터를 포기하고 그로버 알고리즘 전용으로 좁히면** 어디까지 줄일 수 있는지를 끝까지 밀어붙입니다. 핵심은 네 가지입니다 — 진폭을 복소수가 아닌 **실수**로만 들고, 그 실수를 **고정소수점(fixed-point)** Q1.30으로 표현하며, 확산 연산자(diffusion operator)의 행렬-벡터 곱을 **누산 + 비트시프트 + 뿔셈**으로 분해하고, 측정(measurement)의 제곱·나눗셈을 **선형 근사 + 시프트**로 대체합니다.

이 최적화를 담은 Grover Accelerator를 RISC-V(ORCA) SoC에 APB 주변장치로 붙여 **외부 CPU 없이 혼자 도는(standalone)** 에뮬레이터를 완성했고, Kintex Ultrascale+에서 기존 방식 한계 4큐비트를 **6큐비트**로, Arty A7에서 2큐비트를 **4큐비트**로 끌어올렸습니다(LUT 최대 **95.48%** 절감). 4큐비트 기준 최적화된 소프트웨어 대비 **약 191배** 빠릅니다.

이 프로젝트에 가장 직접적인 청사진입니다. 저자가 같은 중앙대 연구실이고, 레포의 project1/hardware/grover_ip.v 가 이미 이 논문의 FSM 구조를 거의 그대로 따르고 있습니다.

목차

- 1. 서론 — 왜 “standalone” 인가
- 2. 예비지식 — 큐비트, 게이트, 그로버
- 3. 자원 최적화 기법 네 가지
 - 3.1 실수 표현으로 충분하다
 - 3.2 고정소수점 표현
 - 3.3 행렬-벡터 곱 분해 — 이 논문의 심장
 - 3.4 측정 근사

- 4. 에뮬레이터 설계와 구현
 - 4.1 SoC 아키텍처와 설계 자동화(QEX)
 - 4.2 Grover Accelerator
- 5. 평가
- 6. 결론
- 이 프로젝트에 어떻게 쓰이나

1. 서론 — 왜 “standalone” 인가

원문 p.2-4

논문의 출발점은 익숙한 진단입니다. 실제 양자컴퓨터는 확장성과 오류정정 문제로 아직 실용 단계가 아니고, 그 공백을 고전 자원 기반 **QC 에뮬레이터**가 메웁니다. 에뮬레이터는 양자회로·알고리즘·응용을 빠르게 실험하고 프로토타이핑하는 통로입니다. 예전에는 대형 고성능 장비에 의존했지만, 요즘은 비용 대비 성능과 재구성 가능성 때문에 FPGA가 주력이 되었습니다.

여기서 이 논문만의 문제의식이 나옵니다. 기존 FPGA 기반 에뮬레이터는 **외부 고성능 CPU와 큰 메모리를 전제한 가속기 모듈**로 설계됩니다. 그런데 양자 알고리즘의 응용처가 IoT 엔드노드, 임베디드 시스템, 웨어러블 같은 저전력·저성능 기기로 확장될 것이라면, 그런 기기에는 외부 CPU에 기대지 않고 **CPU·메모리· 주변장치·QC 전용 가속기를 한 칩에 담아 혼자 도는** 형태가 필요합니다. 그런데 상태와 연산을 벡터·행렬로 들고 있는 정공법은 n 이 조금만 늘어도 자원을 지수적으로 먹어서 저가 FPGA에는 애초에 안 들어갑니다. 고가 FPGA를 써도 한계는 금방 옵니다.

그래서 논문은 목표를 **좁힙니다**. 범용 QC 에뮬레이터가 아니라 **그로버 알고리즘 전용**으로 갑니다. 이유는 두 가지입니다. 하나는 그로버가 정렬되지 않은 데이터 검색을 $O(\sqrt{N})$ 에 해내는, 임베디드 환경에서 쓸모가 클 알고리즘이라는 것(고전은 $O(N)$, 그리고 이 $\sqrt{N}$ 이 최적임이 증명되어 있습니다). 다른 하나는 **그로버의 G 연산이 가진 특수한 성질을 하드웨어에서 내놓고 악용할 수 있다**는 것입니다. 서론에서 미리 밝히는 그 성질들이 곧 3장의 뼈대입니다.

- G 는 실수 행렬이라, 초기 상태벡터가 실수이면 끝까지 실수로 남습니다.
- G 는 오라클(oracle) 행렬과 확산 행렬의 곱으로 분해되고, 확산 행렬을 벡터에 적용하는 일은 **완전한 행렬-벡터 곱 없이** 더 단순한 기본 연산으로 계산됩니다.
- G 를 반복하면 정답 기저상태의 진폭(amplitude)이 1에 가까워지므로, 확률 계산을 **근사**해도 됩니다.

여기에 그로버 특화가 아닌 일반 최적화로 **고정소수점 표현**을 엮습니다. 자원 최적화가 왜 성능 향상이기도 한지도 서론이 짚습니다 — 그로버의 검색 효율은 시스템 규모에 비례하므로, 같은 자원으로 **더 큰 그로버를 올릴 수 있게 만드는 것 자체가** 에뮬레이터의 성능을 올리는 일이기 때문입니다.

성과는 서론에서 이미 다 공개됩니다. 4큐비트 기준 소프트웨어 대비 약 191배 빠른 Grover Accelerator를 설계해 RISC-V에 통합했고, 자원이 넉넉한 Kintex Ultrascale+에서 기존 방식은 4큐비트가 한계였던 것을 **6큐비트**까지, 자원이 빠듯한 Arty A7에서는 2큐비트 한계를 **4큐비트**까지 올렸습니다.

2. 예비지식 — 큐비트, 게이트, 그로버

원문 p.4-6

이 장은 이미 [study_references 1~9장](#)에서 다룬 내용이라 아주 짧게만 정리합니다. 원문을 읽을 때 **기호를 맞춰 두는 것**이 목적입니다.

2.1 큐비트와 양자 게이트

단일 큐비트의 중첩(superposition) 상태는 열벡터로 씁니다 (원문 Eq. 2.1):

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle = \begin{bmatrix} \alpha \\ \beta \end{bmatrix}$$

n 개 큐비트를 이으면 기저상태 수는 $N = 2^n$ 이고, 상태벡터는 N 차원이 됩니다 (원문 Eq. 2.2):

$$|\psi\rangle = \sum_{i=0}^{N-1} \alpha_i |i\rangle = \begin{bmatrix} \alpha_0 \\ \alpha_1 \\ \vdots \\ \alpha_{N-1} \end{bmatrix}, \quad \sqrt{\sum_{i=0}^{N-1} \|\alpha_i\|^2} = 1$$

측정 확률은 계수의 노름 제곱입니다 (원문 Eq. 2.3):

$$P(|\psi\rangle \longrightarrow |i\rangle) = |\langle i|\psi\rangle|^2 = \|\alpha_i\|^2$$

그리고 확률의 성질상 상태벡터의 노름은 항상 1입니다 (원문 Eq. 2.4). **이 Eq. (2.4)가 3.2절에서 “정수부는 1비트면 충분하다”는 결론의 근거가 되니 기억해 두세요.**

게이트는 유니터리(unitary) 행렬이고, 게이트 적용은 행렬-벡터 곱입니다. 원문 Table 1(p.5)이 H, Pauli X/Y/Z, Phase-shift S/T, CNOT의 행렬을 정리해 두었는데, 그로버에 실제로 쓰이는 것은 아다마르(Hadamard)뿐입니다:

$$H = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}$$

여러 큐비트에 게이트를 동시에 걸면 크로네커 곱으로 합칩니다 (원문 Eq. 2.5): $U = U_1 \otimes U_2 \otimes \cdots \otimes U_n$.

원문 Table 1의 Pauli Y 는 $\begin{bmatrix} 0 & -i \\ i & 1 \end{bmatrix}$ 로 인쇄되어 있는데, (2,2) 성분은 0 이 맞습니다(오타). 그로버에는 쓰이지 않아 이후 논의에 영향은 없습니다.

2.2 그로버 알고리즘

찾을 대상은 $f(s) = 1$ 을 만족하는 유일한 s 입니다 (원문 Eq. 2.6). 균등중첩에서 시작해 G 를 k 번 반복하며, k 는 (원문 Eq. 2.7)

$$k = \left\lceil \frac{\pi\sqrt{N}}{4} \right\rceil$$

입니다. 여기서 $\lceil \cdot \rceil$ 은 반올림입니다(내림이 아닙니다 — 4.2절에서 이게 문제가 됩니다). 알고리즘 전체는 (원문 Eq. 2.8)

$$|\psi\rangle = G^k H^{\otimes n} |0^n\rangle, \quad \text{where } G = (H^{\otimes n} Z_{or} H^{\otimes n})(Z_f)$$

오라클 Z_f 는 정답 기저상태의 위상만 뒤집습니다 (원문 Eq. 2.9):

$$Z_f|x\rangle = (-1)^{f(x)}|x\rangle = \begin{cases} -|x\rangle & \text{if } x = s \\ |x\rangle & \text{if } x \neq s \end{cases}$$

균등중첩을 $|n\rangle$ 으로 표기하면 (원문 Eq. 2.10) $|n\rangle = H^{\otimes n}|0^n\rangle = \frac{1}{\sqrt{N}} \sum_{i=0}^{N-1} |i\rangle$ 이고, 확산 연산자는 (원문 Eq. 2.11)

$$H^{\otimes n} Z_{or} H^{\otimes n} = 2|n\rangle\langle n| - I_n = \begin{bmatrix} 2^{-n+1} & \dots & 2^{-n+1} \\ \vdots & \ddots & \vdots \\ 2^{-n+1} & \dots & 2^{-n+1} \end{bmatrix} - I_n$$

여기서 Z_{or} 은 $|0^n\rangle$ 만 남기고 나머지 위상을 뒤집는 연산입니다 (원문 Eq. 2.12).

Eq. (2.11)의 모양을 잘 보세요. 모든 성분이 2^{-n+1} 로 똑같은 행렬에서 단위행렬을 뺀 꼴입니다. 이 “전부 같은 상수”라는 구조가 3.3절에서 행렬-벡터 곱을 통째로 날려버리는 근거가 됩니다. 이게 9장에서 배운 평균 반사의 행렬 표현입니다.

3. 자원 최적화 기법 네 가지

원문 p.6-20

FPGA의 병렬 하드웨어 구조는 중첩과 얽힘(entanglement)에서 오는 자연스러운 병렬성을 흉내 내기에 잘 맞습니다. 문제는 큐비트가 늘면 저장·처리할 기저상태 수가 지수적으로 늘어, 상태와 연산을 벡터·행렬로 두는 전통적 접근이 FPGA 자원 한계에 금세 부딪힌다는 점입니다. 이를 뚫기 위해 네 개의 하위 기법을 제안합니다.

원문은 각 기법마다 Algorithm 1_{6과 Theorem 1}8(정확성 증명)을 붙여 두었습니다. 증명 자체는 “실수 덧셈은 잘 정의되어 있다” 수준의 형식적 확인이라 여기서는 결론과 연산 횟수만 옮깁니다 — 연산 횟수가 이 논문의

실질입니다.

3.1 실수 표현으로 충분하다

원문 p.7-9

일반적인 QC 에뮬레이터는 계수를 복소수로 들기 때문에 실수부·허수부 각각에 레지스터 블록을 할당합니다. 레지스터 소모가 **두 배**입니다. 연산 횟수도 늘어납니다. 원문 Fig. 1(p.7)은 이를 레지스터 수준 데이터패스로 보여줍니다 — (a) 복소수 덧셈은 $\alpha_{Re} + \beta_{Re}$, $\alpha_{Im} + \beta_{Im}$ 두 개의 덧셈기가 나란히 놓인 그림이고, (b)와 (c)는 복소수 곱셈의 실수부·허수부를 각각 만드는 그림으로, 곱셈기 두 개가 교차 배선으로 입력을 받아 덧셈기 하나로 합쳐지는 구조입니다. (c)에서 배선이 X자로 교차하는 것이 $\alpha_{Re}\beta_{Im} + \alpha_{Im}\beta_{Re}$ 를 만드는 부분입니다.

정리하면 이렇습니다.

	복소수 (원문 Algorithm 1)	실수 (원문 Algorithm 2)
덧셈	실수 덧셈 2회	실수 덧셈 1회
곱셈	실수 곱셈 4회 + 실수 덧셈 2회	실수 곱셈 1회 , 덧셈 0회

그런데 **그로버에서는 계수가 끝까지 실수로 남습니다**. 복소수 계수가 가질 수 있는 위상이 0 아니면 π 뿐이고, 그건 실수 계수의 **부호**로 표현되기 때문입니다. G 를 구성하는 기본 연산 — H (Table 1), Z_{or} (Eq. 2.12), Z_f (Eq. 2.9) — 이 전부 실수 행렬이므로, 계수를 실수로 줄여도 **정보 손실이 전혀 없습니다**. 따라서 허수부 레지스터를 아예 두지 않습니다.

시간복잡도는 Algorithm 1도 2도 $O(1)$ 로 같습니다. 논문이 노리는 것은 복잡도 차수가 아니라 **연산 하나당 기본 연산 개수**를 줄이는 것이고, 그것이 반복 실행되는 에뮬레이터의 총 연산량과 연산기·레지스터 자원을 함께 줄입니다. 이 “복잡도가 아니라 상수를 깎는다”는 태도가 3장 전체를 관통합니다.

3.2 고정소수점 표현

원문 p.9-13

실수로 줄였으면 다음 질문은 “그 실수를 어떻게 담을 것인가”입니다. 자연스러운 후보는 단정밀도 부동소수점(floating-point)입니다. 원문 Fig. 2(p.11)가 IEEE 754 형식을 보여줍니다 — 32비트를 부호 S (31번), 지수 E (30₂₃번), 가수 M (22₀번)으로 나누고 값은 $\alpha_i = (-1)^S \times 2^{E-127} \times (1.m_{22}m_{21} \dots m_1m_0)$ 입니다.

문제는 부동소수점 연산의 오버헤드입니다. 원문 Algorithm 3이 FLOAT_ADD와 FLOAT_MULT를 의사코드로 펼쳐 보이는데, 지수 정렬(exponent alignment), 가수 덧셈/뺄셈, 정규화(normalization), 반올림, 재조합의 단계를 거칩니다. 세어 보면 이렇습니다.

	부동소수점 덧셈	부동소수점 곱셈
실수 덧셈	4	4
실수 곱셈	0	1
비교	4	4

	부동소수점 덧셈	부동소수점 곱셈
비트시프트	3	2
XOR	0	1

덧셈 하나 하려고 비교 4번과 시프트 3번이 달려옵니다. 비교는 곧 조건분기(if-else) 하드웨어이고, 정규화 루프(while highest bit = 0)는 가변 지연입니다. 실행시간과 자원 모두에 나쁩니다.

대안이 고정소수점입니다. 부호 비트를 뺀 비트열을 정수부(int.)와 소수부(fraction)로 나누는데, 정수부에 비트를 많이 주면 표현 범위가 넓어지고 소수부에 많이 주면 해상도가 높아지는 트레이드오프입니다. 여기서 Eq. (2.3)이 결정적입니다 — **기저상태 계수의 절댓값은 항상 1 이하**입니다. 그러니 정수부는 **1비트면 충분**하고, 남은 비트를 전부 소수부에 몰아줄 수 있습니다.

또 하나, 고정소수점은 **2의 거듭제곱 곱셈**에서 압도적입니다. 곱셈은 하드웨어에서 비싼 연산입니다 — 가장 단순한 직렬 곱셈기는 i 비트 정수를 곱하는 데 i 클럭이 걸리고, 이를 줄이려면 곱셈기 자체를 키워야 합니다. 부동소수점 곱셈은 가수 정수곱에 더해 정규화·반올림·예외처리 하드웨어까지 붙습니다. 반면 고정소수점에서 2^l 곱하기는 **비트시프트** 한 번이고, 시프트는 배선만 바꾸면 되니 조합회로로 **1클럭**에 끝납니다.

그래서 최종 제안은 이렇습니다. 원문 Fig. 3(p.12)처럼 32비트를 **S 1비트 + Int. 1비트 + fraction 30비트**로 자릅니다. 해상도는 $2^{-30} \approx 10^{-9}$ 입니다. 흔히 쓰는 표기로는 **Q1.30**, 즉 $1.0 = 0x4000_0000$ 입니다.

원문 Algorithm 4의 결론:

	FIXED_ADD	FIXED_MULT_2_EXP ($a \times 2^k$)
실수 덧셈	1	0
비트시프트	0	1

부동소수점의 덧셈 4 + 비교 4 + 시프트 3이 **덧셈 1회**로, 곱셈 일체가 **시프트 1회**로 내려앉았습니다.

3.3 행렬-벡터 곱 분해 — 이 논문의 심장

원문 p.14-16

n 큐비트 시스템에서 게이트 하나를 적용하려면 $N \times N$ 행렬과 $N \times 1$ 벡터를 곱해야 하고, 이는 실수 곱셈 N^2 회 + 실수 덧셈 $N(N-1)$ 회입니다. $N = 2^n$ 이니 게이트 하나가 $O(2^{2n})$ 입니다. n 이 1 늘 때마다 연산량이 **4배**입니다. 게다가 그로버의 G 는 k 번 반복됩니다.

G 를 정공법으로 짜면 $H^{\otimes n}$, Z_{or} , $H^{\otimes n}$ 세 개의 기본 게이트 각각에 행렬-벡터 곱이 필요하므로 (원문 Algorithm 5의 CONV_MATRIX_VECTOR_MULT):

- 실수 덧셈 $3N(N-1)$ 회
- 실수 곱셈 $3N^2$ 회

원문의 표현대로 “행렬-벡터 곱에 기반한 범용 에뮬레이션 프레임워크를 standalone FPGA 기반 QC 에뮬레이터에 올리는 것은 거의 불가능“합니다.

그래서 확산 행렬의 구조를 씁니다. Eq. (2.11)에서 확산 행렬은 “전 성분이 2^{-n+1} 인 행렬 — 단위행렬”이었습니다. 오라클을 이미 적용한 상태를 $|\phi\rangle = Z_f|\psi\rangle = [\beta_0 \cdots \beta_{N-1}]^T$ 라 두면 (원문 Eq. 3.1):

$$G|\psi\rangle = H^{\otimes n} Z_{or} H^{\otimes n} |\phi\rangle = \begin{bmatrix} 2^{-n+1} & \cdots & 2^{-n+1} \\ \vdots & \ddots & \vdots \\ 2^{-n+1} & \cdots & 2^{-n+1} \end{bmatrix} |\phi\rangle - |\phi\rangle = 2^{-n+1} \begin{bmatrix} \sum_{i=0}^{N-1} \beta_i \\ \vdots \\ \sum_{i=0}^{N-1} \beta_i \end{bmatrix} - \begin{bmatrix} \beta_0 \\ \vdots \\ \beta_{N-1} \end{bmatrix}$$

행렬이 통째로 사라졌습니다. 모든 행이 똑같으므로 곱셈 결과의 모든 성분이 “전체 합 $\times 2^{-n+1}$ ”이라는 같은 값이고, 그러면 남는 일은 세 가지뿐입니다. 원문 Fig. 4(p.15)가 이 분해를 그림으로 정리합니다 — 왼쪽의 Diffusion $H^{\otimes n} Z_{or} H^{\otimes n}$ 블록에서 오른쪽으로 세 개의 박스가 갈라져 나오는데, 각각 Accumulation $\sum \beta_i$, Bit-shift 2^{-n+1} , subtraction $-[\beta_0 \cdots \beta_{N-1}]^T$ 입니다.

1. 누산(accumulation): $\sum_{i=0}^{N-1} \beta_i$
2. 비트시프트: 2^{-n+1} 곱하기 $\rightarrow$ 오른쪽으로 $n - 1$ 시프트
3. 뺄셈: 원래 벡터 빼기

전부 실수 덧셈과 비트시프트뿐입니다. 곱셈이 하나도 없습니다. 원문 Algorithm 5의 PROPOSED_DECOMP_G_OPERATION 이걸 그대로 코드로 옮긴 것입니다:

```
acc ← 0
for i = 0 to N-1: acc ← acc + v[i]           // 누산
acc ← acc >> (n-1)                          // 2^{-n+1} 곱하기
for i = 0 to N-1: result[i] ← acc - v[i]    // 뺄셈
```

연산 횟수 비교가 이 논문에서 가장 중요한 표입니다.

확산 게이트 1회	기존 (CONV_MATRIX_VECTOR_MULT)	제안 (PROPOSED_DECOMP_G_OPERATION)
실수 덧셈	$3N(N - 1)$	$2N$
실수 곱셈	$3N^2$	0
비트시프트	0	1
복잡도	$O(N^2) = O(2^{2n})$	$O(N) = O(2^n)$

$O(N^2)$ 이 $O(N)$ 으로 내려갔고, 곱셈기가 완전히 사라졌습니다. 4큐비트($N = 16$)면 곱셈 768회가 0회, 덧셈 720회가 32회입니다.

이것이 9장의 “평균 반사”를 하드웨어 언어로 다시 말한 것임을 알아두면 좋습니다. 계수들의 평균을 $\bar{\beta} = \frac{1}{N} \sum \beta_i$ 라 두면, $\text{acc} \gg (n - 1)$ 은 $\frac{2}{N} \sum \beta_i = 2\bar{\beta}$ 이고 $\text{acc} - v[i]$ 는 $2\bar{\beta} - \beta_i$ — 정확히 평균 중심 반사입니다. 레포의 grover_ip.v 가 two_mean - amp[i] 로 짜 둔 그 식입니다.

원문 Theorem 6의 증명 서술에는 $\text{result}[i] = \text{acc}/N - v[i]$ 로 나오고 “나눗셈 1회”라고 세는데, Eq. (3.1)과 Algorithm 5의 $\text{acc} \gg (n-1)$ 은 $2/N$ 배입니다. 평균 반사가 성립하려면 $2/N$ 이 맞고,

구현(Algorithm 5, 그리고 4.2절의 RTL)도 $2/N$ 입니다. Theorem 6 본문의 $1/N$ 은 표기 실수로 보이니 Eq. (3.1)을 기준으로 읽으세요.

3.4 측정 근사

원문 p.16-20

G 반복이 끝나면 측정해야 합니다. 정직하게 하려면 Eq. (2.3)대로 N 개 계수를 **제공**해서 확률을 얻고, 그 확률들의 **누적분포**를 만들어야 합니다. 둘 다 비쌉니다 — 제공은 곱셈이고, N 개를 병렬로 제공하려면 곱셈기가 N 개 필요합니다.

3.4.1 측정 확률의 근사 정답이 하나인 $f(x)$ 라면 G 반복 후 $|s\rangle$ 의 계수는 **1에 가까워집니다**. 그러면 x^2 을 $x = 1$ 근처에서 1차 테일러 전개해도 됩니다 (원문 Eq. 3.2):

$$\widetilde{\|\alpha_s\|^2} = 1 + 2(\|\alpha_s\| - 1) = 2\|\alpha_s\| - 1$$

곱셈이 사라지고 **시프트 1번(2배) + 뺄셈 1번**만 남습니다.

나머지 $N - 1$ 개 $|\omega\rangle$ 의 계수는 0에 가까워지고 **서로 값이 같습니다**. 그러니 Eq. (2.4)의 확률 합 = 1을 써서 나눠 가지면 됩니다 (원문 Eq. 3.3):

$$\|\alpha_\omega\|^2 = \frac{1 - \widetilde{\|\alpha_s\|^2}}{N - 1}$$

그런데 $N - 1$ 로 나눠야 합니다. N 이면 시프트인데 $N - 1$ 이라 정수 나눗셈이 필요합니다. 여기서 두 번째 근사가 나옵니다 (원문 Eq. 3.4, 3.5):

$$\frac{1}{N - 1} \approx \sum_{i=1}^p \frac{1}{N^i} \quad \therefore \widetilde{\|\alpha_\omega\|^2} = \left(1 - \widetilde{\|\alpha_s\|^2}\right) \sum_{i=1}^p \frac{1}{N^i}$$

p 는 근사 차수입니다. 이걸 등비급수 $\sum_{i=1}^{\infty} N^{-i} = \frac{1/N}{1-1/N} = \frac{1}{N-1}$ 을 p 항에서 자른 것이라, $p \rightarrow \infty$ 면 **정확해집니다**. 그리고 $N = 2^n$ 이므로 $1/N^i$ 는 전부 시프트입니다. 나눗셈이 **시프트 p 번 + 덧셈 $p - 1$ 번**이 되었습니다.

상대오차는 깔끔하게 떨어집니다 (원문 Eq. 3.6, 3.7):

$$\epsilon = \frac{1}{N - 1} - \sum_{i=1}^p \frac{1}{N^i} = \frac{1}{N - 1} - \frac{N^{p-1} + N^{p-2} + \dots + 1}{N^p} = \frac{N^p - (N^p - 1)}{N^p(N - 1)} = \frac{1}{N^p(N - 1)}$$

$$\therefore \epsilon_r = \frac{\epsilon}{1/(N - 1)} = \frac{1}{N^p}$$

즉 상대오차는 N^{-p} 입니다. n 이 커지면 오차가 지수적으로 **작아진다**는 점이 중요합니다 — 규모를 키울수록 근사가 안전해집니다.

원문은 “4큐비트($N = 16$)에서 $p = 3$ 이면 ϵ_r 은 0.4%”라고 적었는데, $1/16^3 = 0.024\%$ 이고 0.4% 는 $1/16^2$, 즉 $p = 2$ 의 값입니다. 5장에서 실제로 쓴 차수도 $p = 2$ 이고 Table 3도 $p = 2$ 와 맞으니, 이 문장의 $p = 3$ 이 옳기로 보입니다.

3.4.2 누적확률분포 만들기 측정을 흉내 내려면 확률을 가중치로 받아 난수를 뽑는 **가중 난수 생성기(WRNG, weighted random number generator)** 가 필요하고, 그러려면 확률 배열의 **누적합**이 있어야 합니다.

단순하게 앞에서부터 더해 나가면(원문 Fig. 5, p.18) $N - 1$ 클럭이 걸립니다. Fig. 5는 8칸짜리 레지스터 배열이 세 줄로 그려져 있고, 매 줄마다 덧셈기 하나가 한 칸씩 오른쪽으로 옮겨 가며 0, 01, 012, 0123 처럼 누적 범위를 **한 칸씩만** 넓히는 그림입니다. $O(N) = O(2^n)$ 이라 큐비트가 늘면 클럭이 지수적으로 늘어납니다.

제안은 **병렬 누산(parallel accumulation)** 입니다 (원문 Fig. 6, p.18). Fig. 6은 같은 8칸 배열이 acc_stage 0/1/2 세 단계로 내려가는 그림인데, 매 단계에서 **모든 칸이 동시에** 갱신됩니다. 0단계 후에는 각 칸이 01, 12, 23... 처럼 이웃 하나를 흡수하고, 1단계 후에는 012, 0123, 1234... 로 두 칸씩, 2단계 후에는 네 칸씩 흡수해 마지막 줄에서 0123456 까지 완성됩니다. 누적 범위가 **매 단계 2배로** 넓어집니다. 이건 전형적인 **Hillis-Steele prefix sum(스캔)** 구조입니다.

규칙은 이렇습니다. k 번째 단계($k = 0, 1, \dots, n-1$)에서 $i = 2^k, 2^k + 1, \dots, N-1$ 인 모든 레지스터에 대해 **동시에**

$$\text{result}[i] \leftarrow \text{result}[i] + \text{result}[i - 2^k]$$

를 수행합니다. 단계가 끝날 때마다 0번부터 $2^{k+1} - 1$ 번까지의 레지스터가 확정되고, $n - 1$ 단계까지 반복하면 전부 완성됩니다.

	SERIAL_ACC	PROPOSED_PARALLEL_ACC
클럭 수	$N - 1$	n
시간복잡도	$O(N) = O(2^n)$	$O(\log_2 N) = O(n)$

6큐비트면 63클럭이 6클럭입니다. 대신 **매 단계마다 최대 N 개의 덧셈기가 동시에** 필요합니다 — 시간을 자원과 맞바꾼 것이고, 5장에서 LUT가 폭발하는 주된 이유이기도 합니다.

4. 에뮬레이터 설계와 구현

원문 p.20-25

여기부터가 이 프로젝트의 청사진입니다.

4.1 SoC 아키텍처와 설계 자동화(QEX)

원문 p.20-21

원문 Fig. 7(p.21)이 전체 SoC입니다. 그림 한가운데를 **System Interconnect (micro-NoC)** 라는 굵은 가로 버스가 관통하고, 그 위쪽에 두 개의 마스터급 블록 — 왼쪽에 **ORCA Core**(AXI 포트), 오른쪽에 **Grover Accelerator**(APB 포트) — 가 붙습니다. 버스 아래로는 AXI Controller를 거친 **SRAM**, APB로 묶인 **SPI / UART / I2C**, 그리고 또 하나의 APB 그룹으로 **IROM / JTAG / FLASH** 가 매달립니다.

여기서 놓치면 안 되는 설계 결정은 Grover Accelerator가 **AXI 마스터가 아니라 APB 슬레이브**라는 점입니다. 즉 가속기는 SRAM에서 상태벡터를 DMA로 읽어 오지 않습니다. 상태벡터는 가속기 내부 레지스터에 살고, 코어는 APB로 레지스터를 몇 개 쓰기만 합니다. 이게 5장에서 FF는 거의 안 늘고 LUT만 폭발하는 이유와 직결됩니다.

구성요소는 이렇습니다.

- **RISC-V 코어**: ORCA 코어. 저전력이라 IoT/엣지에 맞음.
- **메인 메모리**: 256K SRAM.
- **비휘발성**: Flash.
- **인터커넥트**: 경량 NoC (μ NoC).
- **기타**: 부트/리셋 제어 모듈, UART·SPI·I2C 외부 I/O.
- **동작 주파수**: 50 MHz.
- **RTL 설계 도구**: RISC-V eXpress (RVX).

그리고 **QEX (QC Emulator eXpress)** — RVX를 개조해 만든 **에뮬레이터 RTL 자동 생성기**입니다. 사용자가 NUM_QUBIT 하나만 파라미터로 주면:

1. 기저상태 계수를 담은 레지스터 `amp_state`의 블록 수를 결정 → **NUM_STATE** ($= 2^{\text{NUM_QUBIT}}$)
2. 확산 반복 횟수를 결정 → **MAX_ITERATION** ($= k$, Eq. 2.7)
3. 이 값들에 맞춰 Grover Accelerator의 레지스터와 연산 유닛을 구성
4. Fig. 7의 SoC 전체 RTL을 뽑아 줌

C 언어 API는 딱 두 개입니다.

- `set_oracle` — 오라클이 위상을 뒤집을 기저상태 s 값을 설정
- `activate_grover` — 가속기에 시작 신호 전송

응용 프로그램을 이 API로 짜서 SRAM에 명령어로 올려 두면 ORCA 코어가 실행하고, 에뮬레이션이 끝나면 가속기가 검색 결과를 코어로 돌려줍니다.

4.2 Grover Accelerator

원문 p.21-25

FSM 원문 Fig. 8(p.21)이 FSM입니다. **IDLE, INIT, ORACLE, DIFFUSION, MEASURE** 다섯 상태이고, DIFFUSION은 **DIFF_mac** 과 **DIFF_sub** 두 하위 상태로 나뉩니다. 그림에서 **DIFF_mac + DIFF_sub**는 Diffuser 라는 상자로 묶이고, 그 Diffuser와 ORACLE이 다시 **G operator** 라는 큰 상자로 묶여 있습니다 — $G = (\text{Diffusion})(\text{Oracle})$ 을 그대로 시각화한 것입니다.

전이 조건은 이렇습니다.

```
IDLE --(activate == 1)--> INIT
INIT --(iteration <= 0)--> ORACLE
ORACLE --(iteration++)--> DIFF_mac --> DIFF_sub
DIFF_sub --(iteration < MAX_ITERATE)--> ORACLE // G 한 번 더
DIFF_sub --(iteration == MAX_ITERATE)--> MEASURE
MEASURE --(Search done)--> IDLE
```

원문은 2큐비트($N = 4$) 예제로 각 상태의 레지스터 수준 회로도(원문 Fig. 9, p.22)를 보여줍니다. n 큐비트로 확장하면 똑같은 구조를 $N = 2^n$ 개 레지스터에 적용하면 됩니다.

INIT 원문 Fig. 9a: reg amp_state 의 네 칸 [0]~[3] 에 각각 $1/\sqrt{N}$ 이 그대로 실려 들어가는, 배선만 있는 그림입니다.

activate_grover 를 받으면 amp_state 전체를 그로버의 초기 상태 $|n\rangle = H|0^n\rangle$ 에 해당하는 값으로 채웁니다. Eq. (2.4)에 의해 그 값은 전부 같은 양수 $1/\sqrt{N}$ 입니다.

문제는 **제곱근**입니다. 하드웨어에서 비쌉니다. 그래서 n 의 홀짝으로 갈라 상수 나누기 2의 거듭제곱으로 바꿉니다 (원문 Eq. 4.1):

$$\frac{1}{\sqrt{N}} = \begin{cases} 1 \times \frac{1}{2^{n/2}}, & n \text{ even} \\ \frac{1}{\sqrt{2}} \times \frac{1}{2^{(n-1)/2}}, & n \text{ odd} \end{cases}$$

이걸 Q1.30 상수 시프트로 구현합니다:

```
init_coefficient = (NUM_QUBIT % 2 == 0) ?
    (0x4000_0000 >> (NUM_QUBIT / 2)) :
    (0x2D41_3CCE >> ((NUM_QUBIT - 1) / 2));
```

0x4000_0000 은 Q1.30에서 **1.0**(= 2^{30} , cf. 원문 Fig. 3)이고, 0x2D41_3CCE 는 $1/\sqrt{2} = 0.7071 \dots$ 입니다($0.7071 \times 2^{30} = 759,250,125$, 실제 0x2D413CCE = 759,250,126). 계산해 보면 $n = 4$ 일 때 $0x4000_0000 \gg 2 = 2^{28}$, Q1.30으로 $0.25 = 1/\sqrt{16}$ 로 정확히 맞습니다. **제곱근 회로 없이 상수 하나와 시프트 한 번**으로 초기화가 끝납니다.

ORACLE 원문 Fig. 9b: 맨 위의 reg func_table 이 아래로 뺀어 네 개의 비교기(=)에 들어가고, 각 비교기는 인덱스 상수 0/1/2/3과 비교합니다. 비교 결과가 각 칸의 2:1 맥스(mux) 선택 신호가 되어 -1을 곱한 값과 원래 값 중 하나를 골라 amp_state 로 되씹습니다.

s 값은 알고리즘 시작 전에 set_oracle API로 미리 가속기 내부 **func_table** 레지스터에 저장해 둡니다. ORACLE 상태는 Eq. (2.9)를 그대로 수행합니다:

```
for(i = 0; i < NUM_STATE; i++){
    if(i == func_table) amp_state[i] = -amp_state[i]
```

```

    else                amp_state[i] =  amp_state[i]
}

```

부호 반전뿐이라 **1클럭**입니다. func_table 을 레지스터로 둔 덕분에 재합성 없이 런타임에 검색 대상을 바꿀 수 있습니다.

DIFFUSION 3.3절의 세 단계(누산 → 시프트 → 뺄셈)를 두 상태로 나눕니다.

DIFF_mac (원문 Fig. 9c): reg amp_state 네 칸의 출력이 전부 하나의 Accumulator(원 안에 $\oplus$)로 모이고, 그 결과가 Right Bit-shift by $n-1$ 블록을 거쳐 reg acc_result 에 저장되는 그림입니다. 누산과 2^{-n+1} 곱하기(= 오른쪽 $n - 1$ 시프트)를 담당합니다.

DIFF_sub (원문 Fig. 9d): reg acc_result 가 위쪽 배선을 타고 되돌아와 네 개의 감산기에 **브로드캐스트**되고, 각 감산기가 $\text{acc_result} - \text{amp_state}[i]$ 를 계산해 amp_state 로 되쓰는 그림입니다.

RTL로 옮기면 이렇습니다.

```

acc_result = 0;

// DIFF_mac
for(i = 0; i < NUM_STATE; i++)
    acc_result = acc_result + amp_state[i];
acc_result >> NUM_QUBIT - 1;

// DIFF_sub
for(i = 0; i < NUM_STATE; i++)
    amp_state[i] = acc_result - amp_state[i];

```

for 문으로 쓰여 있지만 하드웨어에서는 **전부 펼쳐져 동시에** 수행됩니다 — DIFF_mac은 N 입력 덧셈 트리 1클럭, DIFF_sub는 N 개 감산기 1클럭입니다. 그래서 G 한 번이 **3클럭**(ORACLE 1 + DIFF_mac 1 + DIFF_sub 1)에 끝납니다. 이것이 5장에서 시간이 거의 안 늘고 LUT만 폭발하는 정확한 이유입니다.

ORACLE·DIFFUSION 반복 iteration 레지스터가 G 실행 횟수를 셉니다. 0에서 시작해 G 마다 1씩 증가하고, MAX_ITERATION 에 도달하면 MEASURE로 넘어갑니다.

MAX_ITERATION 은 Eq. (2.7)의 k 인데, 여기도 제공근이 있습니다. INIT과 같은 수법으로 처리합니다 (원문 Eq. 4.2):

$$\frac{\pi\sqrt{N}}{4} = \begin{cases} \left(\frac{\pi}{4} \times 2^{32}\right) \times 2^{\frac{n}{2}-32}, & n \text{ even} \\ \left(\frac{\pi\sqrt{2}}{4} \times 2^{32}\right) \times 2^{\frac{n-1}{2}-32}, & n \text{ odd} \end{cases}$$

상수 $\frac{\pi}{4} \times 2^{32}$ 와 $\frac{\pi\sqrt{2}}{4} \times 2^{32}$ 는 각각 0x0_C90F_DAA2 와 0x1_1C58_31AC 입니다 (홀수 쪽은 값이 2^{32} 를 넘어 33비트입니다).

```

MAX_ITERATE = (NUM_QUBIT % 2 == 0) ?
    0x0_C90F_DAA2 >> (32 - (NUM_QUBIT / 2)) :
    0x1_1C58_31AC >> (32 - ((NUM_QUBIT - 1) / 2));

```

검산해 보면 Eq. (2.7)의 k 와 맞습니다. $n = 4$ 는 $0xC90FDAA2 \gg 30 = \lfloor 3.14 \rfloor = 3$ 이고 $\lfloor \pi\sqrt{16}/4 \rfloor = 3$, $n = 6$ 은 $\gg 29 = \lfloor 6.28 \rfloor = 6$, $n = 3$ 은 $0x11C5831AC \gg 31 = \lfloor 2.22 \rfloor = 2$ 입니다.

주의할 점 하나. Eq. (2.7)의 $\lfloor \cdot \rfloor$ 은 반올림인데 **오른쪽 시프트는 내림**입니다. $n = 2$ 에서 $\gg 31 = \lfloor 1.57 \rfloor = 1$ 이 나오고, 반올림이었다면 2입니다. 그런데 $N = 4$ 의 진짜 최적 반복은 **1회**입니다 — $\theta = \arcsin(1/\sqrt{4}) = 30^\circ$ 이고 $\sin((2 \cdot 1 + 1) \cdot 30^\circ) = \sin 90^\circ = 1$ 이라 정확히 확률 1에 도달하니까요. Eq. (2.7)이 큰 N 을 위한 근사식이라 작은 N 에서 어긋나는 것이고, 이 경우 내림이 오히려 정답을 줍니다. 레포의 grover_tb.v 가 R_ITERS(1) 로 두고 진폭 (0, 0, 1, 0) 을 기대하는 것과 일치합니다.

MEASURE MAX_ITERATE 만큼 G 를 돌린 뒤 **WRNG 모듈**이 amp_state 를 받아 각 원소를 제공하고 누적해 측정 확률의 누적분포를 만들고, 그걸 가중치로 난수를 뽑습니다. 그 난수가 곧 측정 결과입니다.

WRNG는 3.4절의 두 선형근사(Eq. 3.2, 3.5)를 씁니다. 그러려면 amp_state 의 각 원소가 $|s\rangle$ 인지 $|\omega\rangle$ 인지를 알아야 하는데, **THRESHOLD 상수와 비교**해서 판정합니다. $\text{amp_state}[s] > \text{THRESHOLD}$ 이면 그 칸이 $|s\rangle$ 이고 s 를 s_value 레지스터에 저장합니다. 근사 차수 p 는 **APRX_ORDER** 상수로 조절합니다. 결과는 acc_state 레지스터에 쌓입니다.

```

// calculate prob. for s-state
for(s = 0; s < NUM_STATE; s++) {
    if(amp_state[s] > THRESHOLD) {
        s_value = s;
        acc_state[s] = (amp_state << 1) - 1;    // Eq. (3.2)
    }
}

// calculate prob. for omega-states
for(omega = 0; omega < NUM_STATE; omega++) {
    if(omega != s_value) {
        acc_state[omega] = (1 - acc_state[s_value]) >> NUM_STATE
        + (1 - acc_state[s_value]) >> (NUM_STATE * 2)
        + ...
        + (1 - acc_state[s_value]) >> (NUM_STATE * APRX_ORDER);    // Eq. (3.5)
    }
}

```

그다음 3.4.2절의 병렬 누산으로 acc_state 를 누적분포로 바꿉니다.

```

// accumulate prob.
for(k = 0; k < NUM_QUBIT; k++) {
    for(i = 1 << k; i < NUM_STATE; i++) {
        acc_state[i] = acc_state[i] + acc_state[i - 1 << k];
    }
}

```

}

바깥 k 루프가 단계(순차, n 클럭), 안쪽 i 루프가 병렬입니다.

이 두 코드 조각(원문 p.25)에는 오타가 있습니다. 그대로 옮겨 짜면 동작하지 않으니 짚어 둡니다.

- `acc_state[s] = (amp_state << 1) - 1;` → 인덱스가 빠졌습니다. `amp_state[s]` 여야 합니다.
- `>> NUM_STATE, >> (NUM_STATE * 2), ...` → **NUM_QUBIT** 이어야 합니다. $1/N^i = 2^{-ni}$ 이므로 시프트량은 $n \times i = \text{NUM_QUBIT} * i$ 입니다. $\text{NUM_STATE} = 2^n$ 으로 시프트하면(예: $n = 4$ 일 때 16, 32, 48비트) 값이 통째로 0이 됩니다. 본문 서술과 Eq. (3.5)는 정상이므로 **수식을 기준으로** 읽으세요.
- `acc_state[i - 1 << k]` → C 연산자 우선순위상 $(i-1) << k$ 로 파싱됩니다. 본문이 말하는 인덱스는 $i - 2^k$ 이므로 `acc_state[i - (1 << k)]` 가 맞습니다.

5. 평가

원문 p.26-29

실험 환경

QEX로 큐비트 수를 바꿔 가며 에뮬레이터를 생성하고 두 보드에 프로토타이핑했습니다 (원문 Fig. 10, p.26이 두 보드가 노트북에 물려 UART 콘솔로 검색 결과를 찍고 있는 사진입니다).

- **Kintex Ultrascale+** — 자원이 넉넉한 소비자용 FPGA 대표
- **Arty A7** — IoT/임베디드에 어울리는 저성능 FPGA 대표

비교 대상 **baseline**은 기존 방식, 즉 **행렬-벡터 곱 기반 범용 QC** 에뮬레이터입니다. 코어(ORCA), 메모리, NoC 등은 **전부 동일**하고 **Grover Accelerator만 다릅니다**. 합성은 Vivado입니다.

논문이 한 문단을 들여 강조하는 점이 있습니다 — 자원 사용량은 **이론 추정**이 아니라 **실제 합성 결과**로 비교해야 한다는 것입니다. $O(N^2)$ 이니 n 이 1 늘면 연산량이 4배이고 병렬 처리 하드웨어도 4배 늘 것으로 “예상”되지만, 실제 합성은 Vivado 내부의 독점적 최적화 알고리즘이 타이밍·전력·가용 자원을 보며 결정하므로 이론만으로는 예측이 안 됩니다. 그래서 여러 큐비트 수로 직접 합성해 실측했습니다.

공통 컴포넌트의 자원 소모는 이렇습니다 (원문 Table 2, Kintex Ultrascale+ 기준).

Components	LUT	FF
RISC-V ORCA Core	2225	2183
SRAM	175	314
micro-NoC	4070	6108
Peripheral	926	1055
etc.	1872	5073

합이 대략 LUT 9,268 / FF 14,733입니다. **2큐비트 제안 에뮬레이터 전체가 LUT 10,667 / FF 15,662**이니, 그 시점에서 Grover Accelerator 자체는 LUT 1,400 정도밖에 안 씁니다. 아래 표들을 읽을 때 이 **기저 부담**을

빠고 봐야 가속기의 실제 증가분이 보입니다.

근사 오차

측정 확률 근사로 생기는 최대 오차입니다 (원문 Table 3). $p = 2$ 를 썼습니다.

#Qubits	2	3	4	5	6
Max. error	6.25×10^{-2}	1.56×10^{-2}	3.90×10^{-3}	9.77×10^{-4}	2.44×10^{-4}

값들이 정확히 $N^{-p} = 2^{-2n}$ 입니다 ($n = 2: 2^{-4} = 0.0625, n = 6: 2^{-12} = 2.44 \times 10^{-4}$). 오차의 출처를 논문이 정리해 줍니다.

- 실수 표현과 행렬-벡터 곱 분해는 그로버의 행렬 성질을 이용한 것이라 오차가 전혀 없습니다.
- 고정소수점은 소수부가 유한해 반올림 오차가 생기지만 $2^{-30} \approx 10^{-9}$ 수준입니다.
- 측정 확률 근사는 $N^{-p} = 2^{-np}$ 규모이고, 위 반올림 오차보다 압도적으로 큼니다. 즉 정밀도를 지배하는 건 고정소수점이 아니라 측정 근사입니다.

n 이 커질수록 오차가 지수적으로 작아지므로, 규모를 키울수록 이 근사는 더 안전해집니다.

원문 본문은 “ $p = 2$ 의 근사 오차는 2^{-12} 이하”라고 적었는데, 2^{-12} 는 Table 3에서 $n = 6$ 일 때의 값이고 n 이 작을수록 오차는 커집니다($n = 2$ 는 2^{-4}). “ $n = 6$ 에서 2^{-12} 까지 작아진다” 정도로 읽는 것이 맞습니다.

Kintex Ultrascale+ 자원 사용량 (원문 Table 4)

LUT

# Qubit	Baseline Count	Baseline 가용대비	Proposed Count	Proposed 가용대비	절감률
2	16316	3.12%	10667	2.04%	34.62%
3	34578	6.62%	12006	2.30%	65.28%
4	244959	46.86%	15386	2.94%	93.72%
5	1833047	350.67%	82814	15.84%	95.48%
6	-	-	303993	58.16%	-

FF

# Qubit	Baseline Count	Baseline 가용대비	Proposed Count	Proposed 가용대비	절감률
2	15580	1.49%	15662	1.50%	-0.53%
3	16200	1.55%	15895	1.52%	1.88%
4	16696	1.60%	16606	1.59%	0.54%
5	18089	1.73%	17691	1.69%	2.20%

# Qubit	Baseline Count	Baseline 가용대비	Proposed Count	Proposed 가용대비	절감률
6	-	-	19852	1.90%	-

baseline은 5큐비트에서 LUT가 **가용 자원의 3배를 넘어** 합성 자체가 불가능합니다. 즉 **4큐비트가 한계**입니다. 제안 방식은 6큐비트에서도 58.16%로 들어갑니다. LUT 절감 최대 **95.48%**.

이 표에서 반드시 읽어야 할 것은 절감률이 아니라 LUT와 FF의 비대칭입니다. FF는 2→6큐비트에서 15,662 → 19,852로 겨우 27% 늘었는데, LUT는 10,667 → 303,993으로 **28배** 폭발합니다. 왜 그럴까요.

FF는 상태벡터 저장입니다 — 6큐비트면 $64 \times 32\text{bit} = 2,048$ FF, acc_state 까지 합쳐도 4천 FF 남짓. 정확히 관측된 증가분입니다. **선형**으로 늘니다.

LUT는 **조합 논리**입니다 — N 입력 덧셈 트리, N 개 감산기, N 개 비교기·먹스, WRNG의 N 개 근사 회로, 그리고 병렬 누산의 n 단계 $\times N$ 덧셈기가 **전부 펼쳐져 동시에** 존재합니다. 3.4.2절과 4.2절에서 모든 루프를 1클럭에 밀어 넣은 대가가 여기서 청구됩니다. 논문 스스로 5장 말미에서 이를 “**계산 오버헤드를 ’실행시간’에서 ’하드웨어 자원’으로 옮긴 것**”이라고 표현합니다. 이건 버그가 아니라 **의도된 트레이드오프**이고, 우리가 다른 지점을 고를 수도 있는 지점입니다.

Arty A7 자원 사용량 (원문 Table 5)

LUT

# Qubit	Baseline Count	Baseline 가용대비	Proposed Count	Proposed 가용대비	절감률
2	16166	77.72%	11701	56.25%	27.62%
3	27029	129.95%	13064	62.81%	51.67%
4	-	-	16335	78.53%	-

FF

# Qubit	Baseline Count	Baseline 가용대비	Proposed Count	Proposed 가용대비	절감률
2	16179	38.89%	16078	38.65%	0.62%
3	16546	39.77%	16353	39.31%	1.17%
4	-	-	16989	40.84%	-

baseline은 3큐비트에서 이미 가용 LUT의 129.95%로 넘쳐 **2큐비트가 한계**입니다. 제안 방식은 **4큐비트까지** 올라갑니다(78.53%). Arty A7의 LUT 총량이 약 20,800개이니, 4큐비트가 현실적 상한입니다.

성능 (원문 Table 6)

세 가지를 비교했습니다. 전부 C로 작성했습니다.

- **sw_naive** — Grover Accelerator 없이 RISC-V만으로, $H^{\otimes n}$ 의 행렬 표현을 써서 소프트웨어 행렬-벡터 곱으로 그로버 수행
- **sw_Grover_optimized** — sw_naive에 제안 최적화(고정소수점, 축약된 행렬-벡터 곱) 적용
- **hw_Grover_Accelerator** — Grover Accelerator를 탑재한 에뮬레이터에서, 제안 최적화 전부 적용

NUM_QUBIT	sw_naive	sw_Grover_optimized	hw_Grover_Accelerator
2	16730	10.21	3.65
3	113097	384.91	6.85
4	629452	1358.03	7.10

읽는 법입니다.

- **sw_naive는 실용성이 없습니다.** 최적화 버전 대비 수백~수천 배 느립니다(2큐비트 1,638배, 3큐비트 294배, 4큐비트 463배).
- **sw_Grover_optimized는 큐비트가 늘수록 급격히 느려집니다** ($10.21 \rightarrow 384.91 \rightarrow 1358.03$). $O(N)$ 이 순차 실행이니 당연합니다.
- **hw_Grover_Accelerator는 거의 안 늘어납니다** ($3.65 \rightarrow 6.85 \rightarrow 7.10$). 2→4큐비트에서 2배도 안 됩니다. 하드웨어가 연산 유닛 개수를 늘려 늘어난 계산을 흡수하므로 실행시간이 지수적으로 증가하지 않습니다.
- **4큐비트에서 hw_Grover_Accelerator는 sw_Grover_optimized보다 191배 빠릅니다** ($1358.03/7.10 = 191.3$). 서론과 초록의 “191배”가 이것입니다 — naive 대비가 아니라 최적화된 소프트웨어 대비입니다. naive 대비로는 약 88,600배입니다.

표의 단위는 원문에 “클럭 사이클 수”로 적혀 있는데 소수점이 붙어 있습니다(3.65, 1358.03). 여러 s 값에 대한 평균으로 보이지만 원문이 밝히지 않습니다. 또 4.2절의 FSM대로라면 4큐비트는 INIT 1 + G 3회 × 3클럭 = 최소 10클럭 + MEASURE라 7.10과는 맞지 않으니, **절대 사이클 수보다 비율(191배)을 결론으로 받아들이는 편이 안전합니다.**

6. 결론

원문 p.30

논문이 스스로 정리하는 기여는 이렇습니다. 대규모 양자컴퓨터의 실용성 한계를 메우려 FPGA 기반 QC 에뮬레이터가 등장했지만 자원 요구가 여전히 장벽이었고, 이를 넘기 위해 **그로버에 특화된 최적화 기법**을 제안해 자원 요구를 크게 줄이고 **standalone FPGA 구현**을 가능하게 했습니다. 그 기법을 담은 **Grover Accelerator**를 설계해 **RISC-V SoC 플랫폼**에 통합했고, 고성능 Kintex Ultrascale+와 저성능 Arty A7 **양쪽에서 standalone으로 동작**시켰습니다. 기존 QC 에뮬레이터와의 자원·성능 비교로 우수성을 입증했습니다.

연구비 지원은 KIAT(P0017011), NRF(RS-2024-00345668), 중앙대학교 대학원 연구장학금입니다.

이 프로젝트에 어떻게 쓰이나

레포의 project1/hardware/grover_ip.v 는 이미 이 논문의 축소판입니다. 우연이 아니라 같은 물리적 통찰에서 나온 같은 구조입니다. 대응을 보면 이렇습니다.

논문	grover_ip.v
실수 계수만 저장 (3.1절)	reg signed [DW-1:0] amp [0:N-1]
고정소수점 Q1.30 (3.2절)	Q1.16 (DW=18, FRAC=16)
FSM: IDLE/INIT/ORACLE/DIFF_mac/DIFF_sub/MEASURE	S_IDLE/S_INIT/S_ORACLE/S_SUM/S_DIFF/S_MEAS
amp_state[i] = init_coefficient	amp[i] <= INIT_AMP
if(i == func_table) amp_state[i] = -amp_state[i]	amp[MARK] <= -amp[MARK]
acc >> (NUM_QUBIT-1), acc - amp_state[i]	mean = sum>>>NBITS; two_mean = mean<<<1; two
WRNG + 누적분포 + 가중 난수	argmax of amp[i]

그대로 베낄 가치가 있는 것

1. **확산의 시프트 한 번.** 논문은 $\text{acc_result} \gg (\text{NUM_QUBIT} - 1)$ 로 $2/N$ 을 한 번에 만듭니다. 레포는 $\text{sum} \gg \gg \text{NBITS}$ 로 평균을 만들고 다시 $\text{mean} \ll \ll 1$ 로 2배 합니다 — 두 번 시프트하는 사이에 **하위 비트가 버려집니다**($\gg n$ 으로 잃은 LSB는 $\ll 1$ 로 돌아오지 않습니다). 논문식이 정밀도상 엄격히 우월하고 배선도 하나 줄어듭니다. 한 줄 고치면 되는 무료 개선입니다.

2. **MARK 파라미터 → func_table 레지스터.** 현재 grover_ip.v 는 정답 인덱스가 **파라미터**라 검색 대상을 바꾸려면 재합성해야 합니다. 논문처럼 레지스터로 빼고 set_oracle 로 쓰게 하면 런타임에 바뀝니다. 오라클 회로 자체는 그대로(비교기 + 맥스)이니 비용이 거의 없습니다.

3. **NUM_QUBIT 하나로 파라미터화 (QEX의 아이디어).** 지금은 N 을 바꿀 때마다 INIT_AMP 와 R_ITERS 를 손으로 계산해 넣어야 합니다(grover_tb.v 의 18'sd32768, R_ITERS(1)). 논문의 두 상수 트릭을 local-param 으로 옮기면 자동화됩니다.

```
localparam integer NUM_STATE = 1 << NUM_QUBIT;
localparam signed [31:0] INIT_AMP =
    (NUM_QUBIT % 2 == 0) ? (32'h4000_0000 >> (NUM_QUBIT/2))
    : (32'h2D41_3CCE >> ((NUM_QUBIT-1)/2));
localparam integer MAX_ITERATE =
    (NUM_QUBIT % 2 == 0) ? (33'h0_C90F_DAA2 >> (32 - (NUM_QUBIT/2)))
    : (33'h1_1C58_31AC >> (32 - ((NUM_QUBIT-1)/2)));
```

제공된 회로도, 손계산도 없이 NUM_QUBIT 만 바꾸면 됩니다. grover_min_ip.v 의 INIT_AMP = 20'sd23170 (Q4.16의 $1/\sqrt{8}$) 도 이걸로 대체됩니다. 다만 앞서 짚은 대로 $n = 2$ 에서 MAX_ITERATE = 1 이 나눔에 유의하세요(그리고 그 값이 실제로 맞습니다).

4. **Q1.30으로 넓히기.** 논문의 근거는 명확합니다 — Eq. (2.3)에 의해 계수는 항상 $|\alpha_i| \leq 1$ 이므로 정수부는 1비트면 충분하고, 남은 비트를 전부 소수부에 줍니다. 레포의 Q1.16(해상도 2^{-16})도 4큐비트까진 문제없지만, 큐비트가 늘면 $|\omega\rangle$ 진폭이 작아져 소수부가 부족해집니다. grover_min_ip.v 의 Q4.16(DW=20,

FRAC=16)은 정수부 4비트를 쓰는데, 진폭에는 과합니다 — 논문 논리대로면 그 3비트는 소수부로 가는 게 맞습니다.

어려운 부분 — 여기서 시간이 갑니다

1. 측정이 진짜 측정이 아닙니다. 이게 가장 큰 간극입니다. grover_ip.v 의 S_MEAS 는 $\arg\max |amp[i]|$ — 항상 가장 큰 진폭을 고르는 **결정론적** 선택입니다. 그로버가 확률 알고리즘이라는 사실 자체가 사라집니다. 논문의 MEASURE는 다릅니다.

- 각 진폭을 제공해 확률로 (Eq. 3.2, 3.5의 선형 근사로 곱셈 없이)
- 병렬 누산으로 **누적분포**를 만들고 (3.4.2절)
- **WRNG**로 그 분포에서 난수를 뽑음 → 이게 측정 결과

grover_min_ip.v 에 이미 LFSR이 있으니 난수원은 있습니다. 하지만 **동점 처리용**으로만 쓰이지 Born 규칙 샘플링이 아닙니다. 제대로 하려면 누적분포 배열 + LFSR 난수와의 비교기 사다리(또는 이진 탐색)가 필요합니다. **논문도 이 WRNG를 위해 근사 두 개를 발명해야 했다는** 점이 난이도를 말해 줍니다. 진짜 에뮬레이터를 목표로 한다면 여기가 다음 작업입니다.

2. 병렬 누산 트리. 3.4.2절 구조는 개념은 쉬운데(Hillis-Steele 스캔) RTL에서 n 단계 $\times N$ 덧셈기를 펼치면 LUT를 많이 먹습니다. NUM_QUBIT 으로 generate 하는 걸 권합니다. 단, 처음부터 이걸 할 필요는 없습니다 — 순차 누산($N - 1$ 클럭)으로 **먼저 동작을 맞추고**, 자원 여유를 보고 병렬화하세요. 측정은 알고리즘당 **딱 한 번** 일어나므로, G 반복과 달리 여기서 클럭 몇 개 더 쓰는 건 전체 성능에 거의 영향이 없습니다. 논문이 굳이 병렬화한 건 논문의 기여 목록을 채우기 위한 선택에 가깝습니다.

3. LUT 벽, 그리고 우리가 다른 선택을 할 수 있다는 것. Table 4가 진짜로 말하는 바는 이렇습니다 — 6큐비트에서 LUT 303,993개(58%)를 먹는 이유는 상태벡터가 커서가 아닙니다(FF는 19,852개뿐입니다). **모든 루프를 1클럭에 펼쳤기 때문**입니다. 논문의 선택은 “ G 한 번 = 3클럭”이라는 극단이고, 그 대가가 N 입력 덧셈 트리과 N 개 감산기를 전부 조합회로로 까는 것입니다.

우리는 다른 지점을 고를 수 있습니다. amp_state 를 레지스터 파일 대신 **BRAM**에 두고 누산을 N 클럭에 걸쳐 순차로 하면, LUT는 극적으로 줄고 클럭만 N 배 늘어납니다. 50MHz에서 8큐비트($N = 256$)면 G 하나가 256클럭 $\approx 5\mu s$, $k = 12$ 회면 $60\mu s$ — **여전히 즉각적**입니다. 논문은 APB 슬레이브에 상태를 가둬서(4.1절) 이 선택지를 스스로 닫았습니다. 큐비트 수를 더 밀고 싶다면 **논문을 그대로 따라가면 안 되는 지점**이 바로 여기입니다. 논문의 6큐비트 벽은 알고리즘의 한계가 아니라 **아키텍처 선택의 결과**입니다.

4. SoC 통합은 별개의 프로젝트입니다. ORCA 코어 + μ NoC + APB + RVX/QEX 톨체인은 저자 연구실이 다른 논문 여러 편에 걸쳐 쌓아 온 자산입니다(원문 참고문헌 33~39가 전부 그 계보입니다). grover_ip.v 를 APB 슬레이브로 감싸는 것 자체는 어렵지 않지만, RISC-V SoC 전체를 세우는 건 별도 과제로 분리하세요. 가속기 IP를 **먼저 독립적으로 완성하고** 테스트벤치로 검증한 뒤 붙이는 순서가 맞습니다 — grover_tb.v 가 이미 그 길입니다.

읽는 순서 제안

3.3절(Eq. 3.1)과 4.2절(FSM + 네 상태의 코드)이 **핵심**입니다. 이 둘만 이해하면 grover_ip.v 가 왜 그렇게 생겼는지 전부 설명됩니다. 3.1·3.2절은 근거가 명료하니 빠르게 읽고, Theorem 1~8의 증명은 형식적 확인이라 건너뛰어도 무방합니다. 5장은 Table 4의 **LUT/FF 비대칭**과 Table 6의 **191배**만 챙기면 됩니다.

같은 폴더의 Precision-aware fixed-point emulation of Grover's algorithm 이 3.2절의 고정소수점 정밀도 문제를 훨씬 깊게 파고들 것으로 보입니다. 이 논문이 “ 2^{-30} 이면 충분하고 어차피 측정 근사가 지배한다”고 한 문단으로 넘긴 부분입니다. Q1.16을 유지할지 Q1.30으로 갈지 결정할 때 그 논문을 먼저 보세요.